

XRPL Lending Protocol · Developer Feedback

DeVinci Blockchain × Ripple hackathon · 12 and 13 September 2026

CY-HACK

DevEx pseudonym late-quail-92
github.com/charlyppr/xrpl-lending

TRACK

Track 1, open-ended vault

FLAVOUR

Loaded: XLS-65, XLS-66, TokenEscrow

ENVIRONMENT

Custom Hackathon Devnet, `rippled 3.4.0-rc1` (hackathon branch), `xrpl@4.6.0`
`LendingProtocolV1_1` and `fixCleanup3_4_0` enabled

LIBRARY AND VERSION

The 143 transactions cited are validated on the ledger, type and result code included ([_probe-audit-hashes.mjs](#)). Documentation claims are checked against XLS-65, XLS-66 and `xrpl.org`, library claims against the `rippled` and `xrpl.js` source. Hashes link to the explorer.

#	Finding	Category	Severity
1	<code>fixCleanup3_4_0</code> is enabled and missing from <code>xrpl.org</code>	client libraries	HIGH
2	A late <code>LoanPay</code> without <code>tfLoanLatePayment</code> returns <code>tecEXPIRED</code>	documentation/tutorials	HIGH
3	First-loss capital covered 0.5 % of a defaulted loan	UX	HIGH
4	<code>xrpl.org</code> omits the sole-holder exception, and zero-valued fields are absent	documentation/tutorials	MEDIUM
5	No read command for loans or brokers	missing primitive	MEDIUM
6	One result code covers causes that need different actions	UX	MEDIUM
7	The hackathon build reverts the V1.1 <code>LoanBrokerSet</code> restriction	documentation/tutorials	MEDIUM
8	Vault shares carry <code>lsfMPTCanTrade</code> , but <code>OfferCreate</code> rejects them	documentation/tutorials	MEDIUM
9	No flag closes a vault to deposits	missing primitive	LOW

High: blocks a minimum bar step, or misstates a depositor's risk. **Medium:** costs time or a wrong design, with a workaround. **Low:** documented elsewhere, or cosmetic.

The brief's questions

Vault, broker, loan	Clear. Only the vault owner can submit <code>LoanBrokerSet</code> . The drawdown has no transaction: <code>LoanSet</code> moves the principal (<code>D19CD59F...</code>).
First-loss parameters	Not as named (3).
Multi-party <code>LoanSet</code>	Broker signs, borrower countersigns and submits. The <code>xrpl@4.6.0</code> helper fails here (1).
SDK or raw JSON	All 15 XLS-65/66 types are typed. No raw JSON.
Position and yield	Computed client-side from five calls (5).
Docs and explorer	The docs differ in 1, 2, 4, 8. The explorer hides a flag name (2) and <code>Vault</code> fields (4).

- The broker signs a `LoanSet`, the borrower applies `signLoanSetByCounterparty` and submits: `Counterparty: Invalid signature`, a local check with no hash.
- The same blob signed over `encodeForSigningCounterparty` (`raw-submit.mjs`) returns `tesSUCCESS`.

Transactions impairment `5486020B...`, countersigned `LoanSet 23631C36...`

Proposed fix List `fixCleanup3_4_0` on *Known Amendments* with both lending changes. Update *Manage a Loan*: impairment is accepted once `NextPaymentDueDate` has passed. In `xrpl.js`, sign `CounterpartySignature` with `encodeForSigningCounterparty`, keeping the previous prefix as an option for networks without the amendment.

1 `fixCleanup3_4_0` is enabled and missing from `xrpl.org`

HIGH client libraries · `xrpl@4.6.0`

The amendment is active on the devnet and absent from *Known Amendments*. It changes two lending behaviours, and neither the tutorial nor `xrpl@4.6.0` reflects them.

Impairment. XLS-66, section 3.10.4.2, condition 9 rejects `tfLoanImpair` while `currentTime <= NextPaymentDueDate`. The tutorial *Manage a Loan* still says a broker can impair “before a payment due date passes”.

Counterparty signature. Since `rippled` PR #8162, `CounterpartySignature` is signed with the CPT prefix. `signLoanSetByCounterparty` signs with `encodeForSigning`, the previous prefix, and offers no option.

Repro steps

- `LoanSet` with `PaymentInterval 60` and `GracePeriod 60`. `LoanManage tfLoanImpair` returns `tecT00_SOON` 31 s and 13 s before `NextPaymentDueDate`, and `tesSUCCESS` 5 s after.

2 A late `LoanPay` without `tfLoanLatePayment` returns `tecEXPIRED`

HIGH documentation/tutorials · `xrpl@4.6.0`

Once `NextPaymentDueDate` has passed, every `LoanPay` without the flag fails, even within the grace period: the instalment, the instalment with `LatePaymentFee`, the full balance, and `tfLoanFullPayment`. XLS-66, section 3.11.4.2, condition 11 defines this failure; the `xrpl.org` `LoanPay` reference lists eight error codes, not this one. The explorer shows the flag as `0x00040000`, and no `Loan` field holds the accrued late interest.

Repro steps

- `LoanSet` with `PaymentInterval 90`, `GracePeriod 60`, `PaymentTotal 4` (`_probe-pay-states.mjs`).
- 10 s after the due date, `LoanPay` of `ceil(PeriodicPayment) + LoanServiceFee + LatePaymentFee` returns `tecEXPIRED` with `Flags: 0`, and `tesSUCCESS` with `Flags: 262144`.
- With `LateInterestRate 30 %` (`_probe-h2-race.mjs`), the same amount returns `tecINSUFFICIENT_PAYMENT`. Sending 2 765 001 drops succeeds and takes 765 003.

Transactions no flag [96C38704...](#), flag [EFAD383F...](#), late interest [9A758938...](#)

Proposed fix Add `tecEXPIRED` to the `LoanPay` error table, with `tfLoanLatePayment` as the remedy. Name the flag in the explorer. Expose the amount due now, late interest included, in a `Loan` field or a read command.

❗ Fail - TecEXPIRED

Status
This transaction failed with a status code of `tecEXPIRED`, and validated in ledger `67844` on `September 12, 2026 at 2:33:42 PM UTC`

✅ Success

Flags
`0x00040000`

Explorer. Same loan and amount, without the flag, then with it.

3

First-loss capital covered 0.5 % of a defaulted loan

⚠ HIGH

UX · xrp1@4.6.0

Two brokers, both with `CoverRateMinimum` 10 % and `CoverRateLiquidation` 5 %:

Vault	Loan	Cover	Taken	Loss	Share value
200 XRP	50	50	0.25	49.75	1.000 → 0.751
5 XRP	4	1	0.02	3.98	1.000 → 0.204

The cover taken is $\text{debt} \times 10\% \times 5\%$, to the drop. This is the documented formula: `CoverRateLiquidation` applies to the minimum cover, not to the defaulted debt, and the `xrpl.org` example covers 1 % of the debt. After the default `DebtTotal` is zero, so is the cover floor, and the broker withdrew the rest. Second vault, UTC:

14:23:50	<code>VaultWithdraw: tecINSUFFICIENT_FUNDS</code>	D91BA47E...
14:27:01	<code>tfLoanDefault: 0.02 XRP from the cover</code>	923F7D61...
14:27:11	<code>VaultWithdraw: 1.02 XRP</code>	3338BA62...
14:27:22	<code>LoanBrokerCoverWithdraw: 0.98 XRP</code>	68DD97D9...

Repro steps

- 1 `h1-default.mjs`, after `setup-accounts.mjs`: a 200 XRP vault, 50 XRP of cover, one 50 XRP loan, defaulted after `GracePeriod`.
- 2 Compare `CoverAvailable` and `AssetsTotal` before and after the default.
- 3 `LoanBrokerCoverWithdraw` of the remaining cover returns `tesSUCCESS`.

Transactions first vault default [91F458E3...](#)

Proposed fix Next to the formula, state the share of a defaulted debt the cover absorbs: $\text{CoverRateMinimum} \times \text{CoverRateLiquidation}$, 0.5 % at 10 % and 5 %. Hold `LoanBrokerCoverWithdraw` for a set delay after a default, or state that no delay exists.

4

xrpl.org omits the sole-holder exception, and zero-valued fields are absent

ⓘ MEDIUM

documentation/tutorials · xrp1@4.6.0

XLS-65 redeems shares at $(\text{AssetsTotal} - \text{LossUnrealized}) / \text{total shares}$, except for the sole outstanding holder,

who is not charged `LossUnrealized`. The `xrpl.org` Single Asset Vault page gives the formula without the exception. `LossUnrealized` is absent until an impairment writes it, as are `AssetsAvailable`, `AssetsMaximum`, `PaymentRemaining` and `TotalValueOutstanding` at zero. The explorer's *Detailed* tab shows no field of the `Vault` node.

Repro steps

- 1 `vault_info` on a vault without an impaired loan: no `LossUnrealized` field.
- 2 `tfLoanImpair` on one of its loans, after `NextPaymentDueDate`.
- 3 `vault_info` returns `LossUnrealized "3000001"` with `AssetsTotal "5000002"`. Compare the explorer's *Detailed* and *Raw* tabs.

Transactions impairment [C8678C1C...](#)

Proposed fix Add the sole-holder exception to the `xrpl.org` formula. Return zero-valued fields as `"0"` in `vault_info`, or document their absence. Show changed field values in the explorer's *Detailed* tab.

✅ Success

- It modified a node with type `Loan`
- It modified a node with type `Vault`

`AssetsAvailable: "2000001"`
`AssetsMaximum: "100000000"`
`AssetsTotal: "5000002"`
`Flags: 0`
`LEVersion: 1`
`LossUnrealized: "3000001"`

Explorer. The impairment: *Detailed* tab, then *Raw* tab with the `Vault` node expanded.

5

No read command for loans or brokers

ⓘ MEDIUM

missing primitive · xrp1@4.6.0

A depositor's position takes five calls, then six values computed client-side in `lib/nav.mjs`: gross and net share value, position value, withdrawable amount, utilisation, broker debt capacity. `mpt_holders` is documented as Clio-only, and the devnet serves `rippled`.

Call	Returns
<code>vault_info</code>	totals, share issuance and flags
<code>account_objects mptoken</code>	the depositor's shares
<code>account_objects, owner</code>	the broker, filtered on <code>VaultID</code>
<code>account_objects loan</code>	the loans, on the broker's pseudo-account
<code>mpt_holders</code>	<code>unknownCmd</code>

Repro steps

- 1 Send `loan_info` and `loan_broker_info` to the devnet JSON-RPC endpoint: `unknownCmd`. Neither has a documentation page.

Proposed fix Add `loan_broker_info` and `loan_info`, or list a vault's brokers and loans in `vault_info`. Return share value, withdrawable amount and utilisation. Expose Clio on the devnet.

6 One result code covers causes that need different actions

MEDIUM UX · xrpl@4.6.0

A broker who receives `tecINSUFFICIENT_FUNDS` on `LoanSet` cannot tell whether to add cover or wait for deposits. Three other codes share the problem.

Code	Cause	
tecINSUFFICIENT_FUNDS	LoanSet above the cover limit	5DC3A6E0...
	LoanSet above the vault liquidity	B92A90F5...
	LoanBrokerCoverWithdraw below the floor	4FB4A2B0...
tecLIMIT_EXCEEDED	VaultDeposit above AssetsMaximum	132BAF7C...
	AssetsMaximum set below AssetsTotal	37053F15...
	DebtMaximum set below DebtTotal	82C14ED4...
tecNO_PERMISSION	tfLoanImpair sent by the borrower	F0304194...
	tfLoanUnimpair on an unimpaired loan	13975FD3...
tecNO_AUTH	shares sent to a holder without MPToken	789AAF7E...
	non-transferable shares sent	E9DE504D...

Repro steps

- 1 `_probe-cliffs.mjs`: a `LoanSet` above the cover limit with ample liquidity, then one above the liquidity with ample cover. Both return `tecINSUFFICIENT_FUNDS`.

Proposed fix Where one transaction type maps two causes to one code (`LoanSet`, `LoanManage`, a `Payment of shares`), return distinct codes or a result message naming the check that failed.

7 The hackathon build reverts the V1.1 LoanBrokerSet restriction

MEDIUM documentation/tutorials · xrpl@4.6.0

The V1.1 documentation restricts `LoanBrokerSet` on open-ended vaults, a rule added by rippled PR #8076. A revert of #8076 reached the lending hackathon branch on 10 September 2026. The other V1.1 rules tested (new fields, date validation, phase locks) are enforced. The event brief does not mention the revert.

Repro steps

- 1 feature: `LendingProtocolV1_1` is enabled.
- 2 `LoanBrokerSet` on an open-ended vault returns `tesSUCCESS`.
- 3 `LoanBrokerSet` on a `VaultKind: 1` vault returns `tesSUCCESS` (`_probe-h13-closed.mjs`).

Transactions open-ended `781F54B5...`, closed-ended `1961BE21...`

Proposed fix State in the event brief which V1.1 rules the hackathon build enforces, and that the #8076 restriction is reverted.

8 Vault shares carry `lsfMPTCanTrade`, but `offerCreate` rejects them

MEDIUM documentation/tutorials · xrpl@4.6.0

`VaultCreate` sets `lsfMPTCanEscrow`, `lsfMPTCanTrade` and `lsfMPTCanTransfer` on the share issuance, with no field to

change them. The `xrpl.org` `MPTokenIssuance` reference says `lsfMPTCanTrade` lets holders trade “using the XRP Ledger DEX or AMM”. The devnet has no MPT trading amendment, only `MPTokensV1` and `DynamicMPT`.

Repro steps

- 1 `vault_info`: `shares.Flags` is 56, that is 0x08, 0x10 and 0x20.
- 2 `offerCreate` by a holder, `TakerGets` the share MPT, `TakerPays` XRP: `temDISABLED`, a local rejection with no hash (`_probe-shares.mjs`).

Proposed fix State on the `MPTokenIssuance` reference that DEX trading of MPTs needs an amendment not yet available, or leave `lsfMPTCanTrade` unset on vault shares until then.

9 No flag closes a vault to deposits

LOW missing primitive · xrpl@4.6.0

`VaultSet` exposes `Data`, `AssetsMaximum` and `DomainID`. The only way to stop deposits is a cap equal to `AssetsTotal`. `AssetsMaximum: 0` means no cap in XLS-65 and the `Vault` entry; the `VaultSet` reference does not repeat it.

Repro steps

- 1 `VaultSet` with `AssetsMaximum` equal to `AssetsTotal` returns `tesSUCCESS`.
- 2 A 1 XRP `VaultDeposit` returns `tecLIMIT_EXCEEDED`.

Transactions deposit refused `132BAF7C...`

Proposed fix Add a `VaultSet` flag that closes a vault to deposits. Repeat that `0` means no cap on the `VaultSet` reference.

Minor issues

Issue	Category	Severity
The Lending Protocol concept page spells <code>PrincipleOutstanding</code> and <code>depositor</code> .	documentation/tutorials	LOW
<code>validateVaultCreate</code> in <code>xrpl@4.6.0</code> accepts any <code>WithdrawalPolicy</code> ; 0, 2, 3, 99 and 255 return <code>temMALFORMED</code> .	client libraries	LOW
The faucet ignores <code>destination</code> and funds a new account instead.	other	LOW

What worked

`tecHAS_OBLIGATIONS` names its cause on the three delete transactions. `VaultDelete` removes third-party `MPToken` objects and refunds their reserve. The cover floor is enforced to the drop, and cover rates cannot change on an existing broker (`temINVALID`). Closed-ended vault phases behave as XLS-65 describes.

Limits. XRP only, so no clawback with an issued asset. Payment intervals of 60 to 90 s. One build. The minimum bar run, the guardrails and the escrow swap are in the [README](#).